國立臺北科技大學
National Taipei University of Technology

資料結構
Data Structure

Lab 05

姓名: 盧敬恩
學號: 114310123

Lab05-Q1and Q2(Complete the following parts in the sample code:)

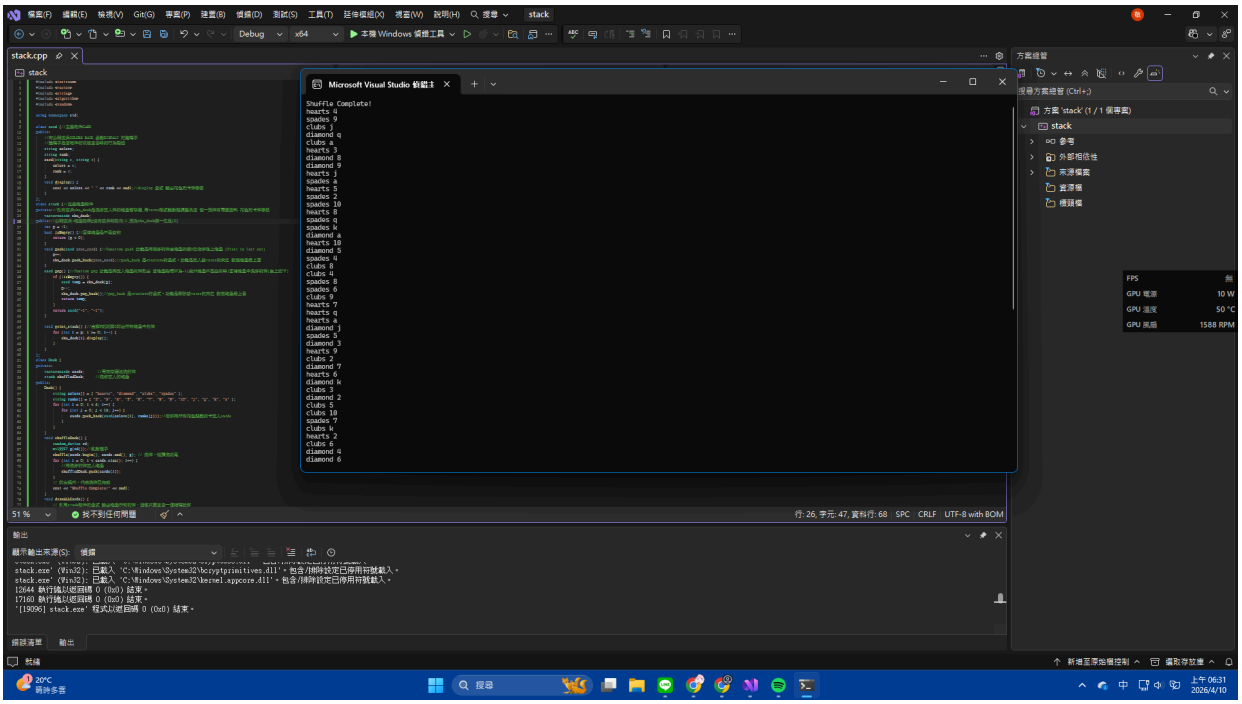
Class Stack with ‘push’, ‘pop’, and ‘isEmpty’- shuffleDeck- drawALLCards

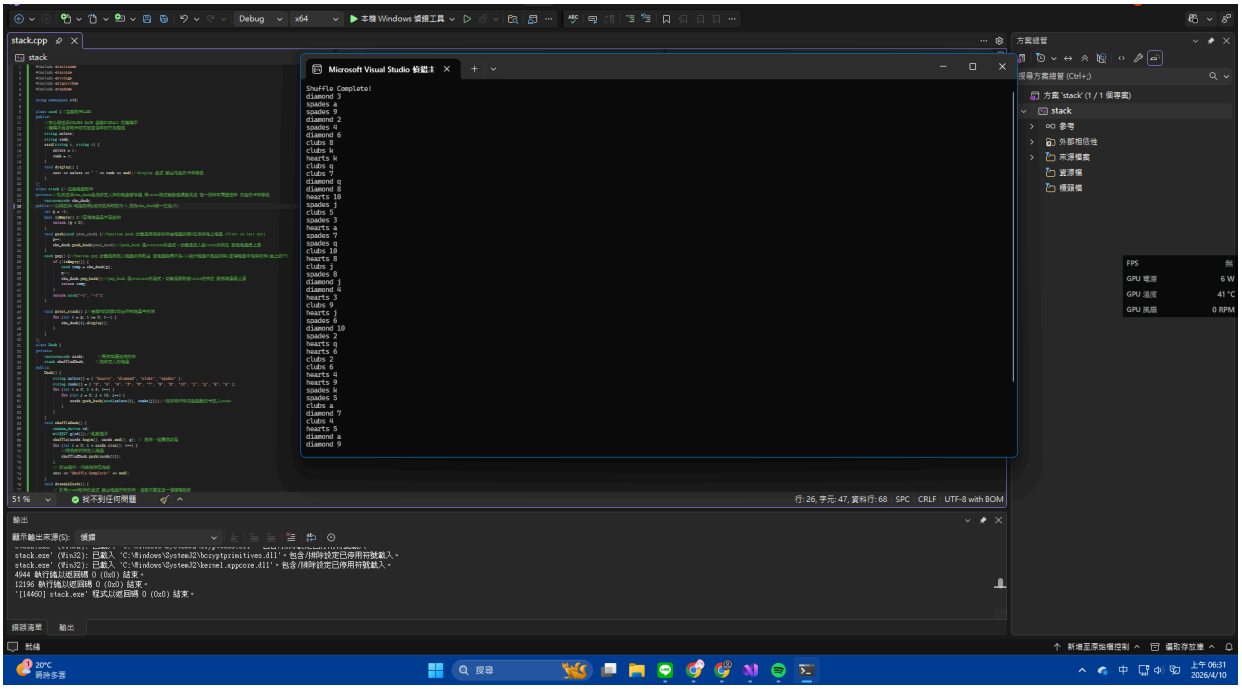
Code

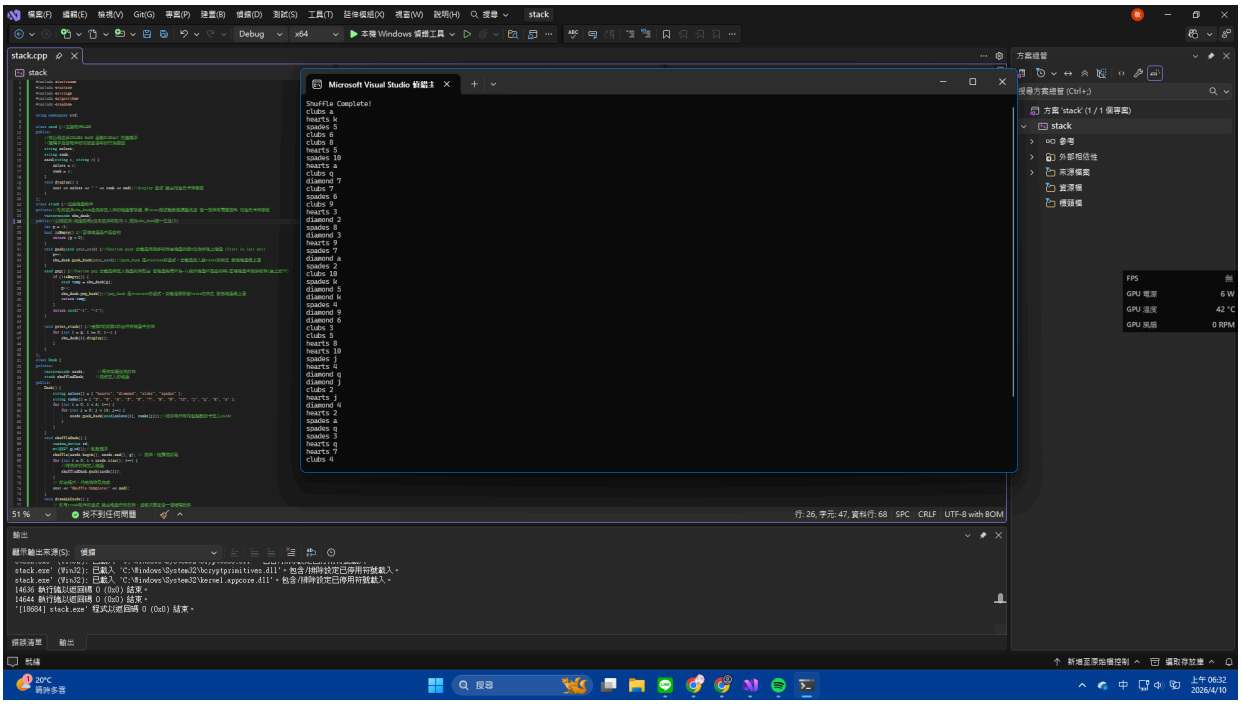
```
#include <iostream>
#include <vector>
#include <string>
#include <algorithm>
#include <random>

using namespace std;

class card {//定義物件CARD
public:
    //有公開成員COLORS RANK 函數DISPLAY 和建構子
    //建構子是當物件初次被宣告時的行為描述
    string colors;
    string rank;
    card(string s, string r) {
        colors = s;
        rank = r;
    }
    void display() {
        cout << colors << " " << rank << endl;//display 函式 輸出花色和卡牌等級
    }
};
class stack {//定義堆疊物件
private://私有成員shu_deck是洗好放入牌的堆疊暫存器,用vetor形式能動態調整長度 每一張牌有兩筆資料 花色和卡牌等級
    vector<card> shu_deck;
public://公開成員:堆疊指標p沒有成員時指向-1,因為shu_deck第一位是[0]
    int p = -1;
    bool isEmpty() {//回傳堆疊是不是空的
        return (p < 0);
    }
    void push(card your_card) {//function push 功能是將洗好的牌由堆疊的第0位依序往上堆疊 (first in last out)
        p++;
        shu_deck.push_back(your_card);//push_back 是<vector>的函式。功能是放入該vector的末位 對應堆疊最上層
    }
    card pop() {//function pop 功能是將放入堆疊的牌取出 當堆疊指標不為-1(表示堆疊不是空的時)回傳堆疊中洗好的牌(由上到下)
        if (!isEmpty()) {
            card temp = shu_deck[p];
            p--;
            shu_deck.pop_back();//pop_back 是<vector>的函式。功能是刪除該vector的末位 對應堆疊最上層
            return temp;
        }
        return card("-1", "-1");
    }

    void print_stack() {//由第P到到第0印出所有堆疊中的牌
        for (int i = p; i >= 0; i--) {
            shu_deck[i].display();
        }
    }
};
```

```
class Deck {
private:
    vector<card> cards;        //用來存還沒洗的牌
    stack shuffledDeck;       //洗好放入的堆疊
public:
    Deck() {
        string colors[] = { "hearts", "diamond", "clubs", "spades" };
        string ranks[] = { "2", "3", "4", "5", "6", "7", "8", "9", "10", "j", "q", "k", "a"
};
        for (int i = 0; i < 4; i++) {
            for (int j = 0; j < 13; j++) {
                cards.push_back(card(colors[i], ranks[j]));//依序將所有花色點數的卡放入cards
            }
        }
    }
    void shuffleDeck() {
        random_device rd;
        mt19937 g(rd());//亂數種子
        shuffle(cards.begin(), cards.end(), g); // 洗牌，從頭洗到尾
        for (int i = 0; i < cards.size(); i++) {
            //將洗好的牌放入堆疊
            shuffledDeck.push(cards[i]);
        }
        // 印出提示，代表洗牌已完成
        cout << "Shuffle Complete!" << endl;
    }
    void drawALLCards() {
        // 引用stack物件的函式 輸出堆疊所有的牌，這樣只要宣告一個結構就好
        shuffledDeck.print_stack();
    }
};
int main() {
    Deck myDeck;//宣告物件MYDECK
    myDeck.shuffleDeck(); // 執行洗牌程式
    myDeck.drawALLCards();  // 輸出洗好的牌
    return 0;
}
```

Result







Discussion

這禮拜的作業我做四個小時非常久，很多時間不是不了解堆疊 而是對於物件導向等語法不熟悉和ai討論了非常久。